

Teste de Software

Prof. Iaçanã Ianiski Weber



Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>

Software Testing

✓ <https://www.cs.virginia.edu/~up3f/cs3250/syllabus.html>

Márcio Eduardo Delamaro, José Carlos Maldonado e Mario Jino. **Introdução ao Teste de Software**. 2007.

✓ <https://www.amazon.com.br/Introdu%C3%A7%C3%A3o-Software-M%C3%A1rcio-Eduardo-Delamaro/dp/8535226346/>

“A qualidade do software está se tornando cada vez mais essencial para todas as empresas e o conhecimento de teste de software está se tornando necessário para todos os engenheiros de software”*

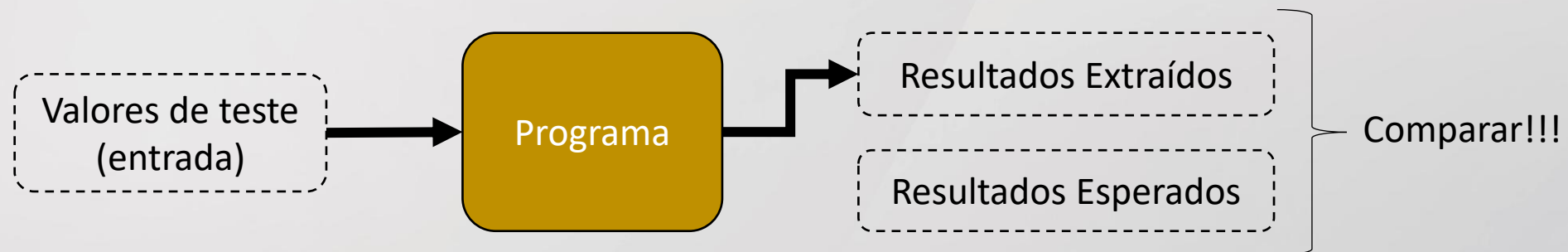
- ✓ *Safety critical*
- ✓ *Real-time software*
- ✓ *Embedded software*
- ✓ *Enterprise applications*
- ✓ *Security*
- ✓ *Web*
- ✓ *Mobile*

*Fonte: Paul Ammann and Jeff Offutt. **Introduction to Software Testing**. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>

- ✓ **Teste:** processo de encontrar entradas para verificar um software
 - Um caso de teste consiste de estímulos de entrada e saídas esperadas
 - Teste somente mostra a presença de falhas, não a ausência
 - Nem todos os estímulos vão causar uma falha !!!
- ✓ **Debug:** processo de encontrar um defeito causador de uma falha

“O verdadeiro objetivo do testador não é realizar os testes, mas se preocupar com o desenvolvimento dos casos de teste!”



*Fonte: Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>

- ✓ Se teste é o processo de **encontrar entradas** para verificar um software, como lidar com a complexidade ?!?!?
- ✓ Mesmo os problemas mais simples possuem um espaço de estados gigante
 - Considere a função para calcular a média entre 3 inteiros
 - *int average(int a, int b, int c)*
 - Cada variável de 32-bits pode ter 4 bilhões de valores possíveis
 - Combinação de C(3,4Bilhões) > **80 octilhões** de testes possíveis !!!

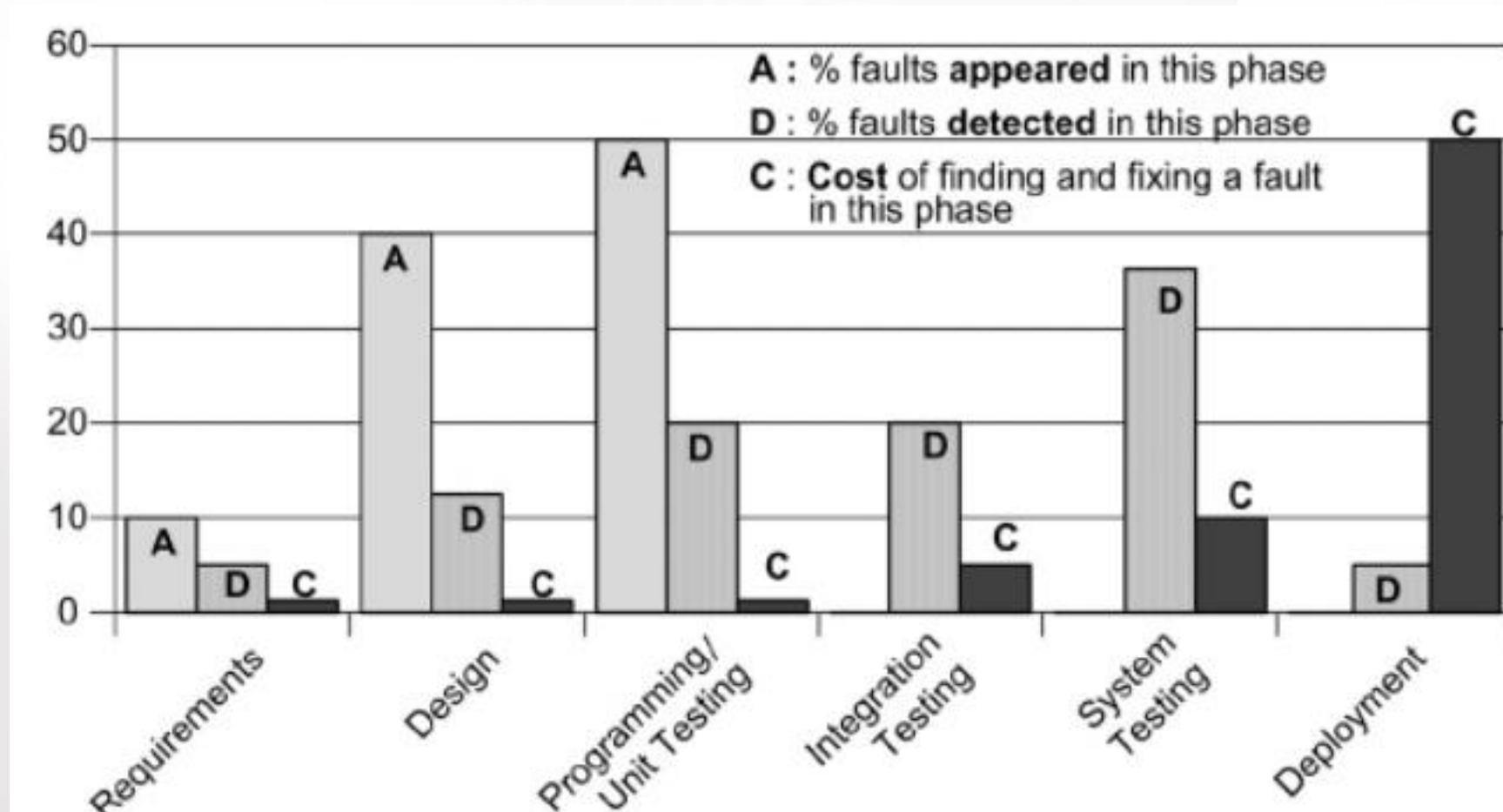
NOME	NÚMERO
Um	1
Dez	10
Cem	100
Mil	1.000
Milhão	1.000.000
Bilhão	1.000.000.000
Trilhão	1.000.000.000.000
Quatrilhão	1.000.000.000.000.000
Quintilhão	1.000.000.000.000.000.000
Sextilhão	1.000.000.000.000.000.000.000
Septilhão	1.000.000.000.000.000.000.000.000
Octilhão	1.000.000.000.000.000.000.000.000.000
Nonilhão	1.000.000.000.000.000.000.000.000.000.000
Decilhão	1.000.000.000.000.000.000.000.000.000.000.000

Considere o teste da função x^y

- ✓ O domínio de entrada é formado pelo par (x,y)
- ✓ Cardinalidade é $2^n * 2^n$
- ✓ Se n for 32 bits, isso representa $2^{64} = 18446744073709551616$
- ✓ Se cada caso de teste for executado em 1 ms, vai levar **5.849.424 séculos** para executar todos os testes!

Fonte: Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>



Fonte: Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>

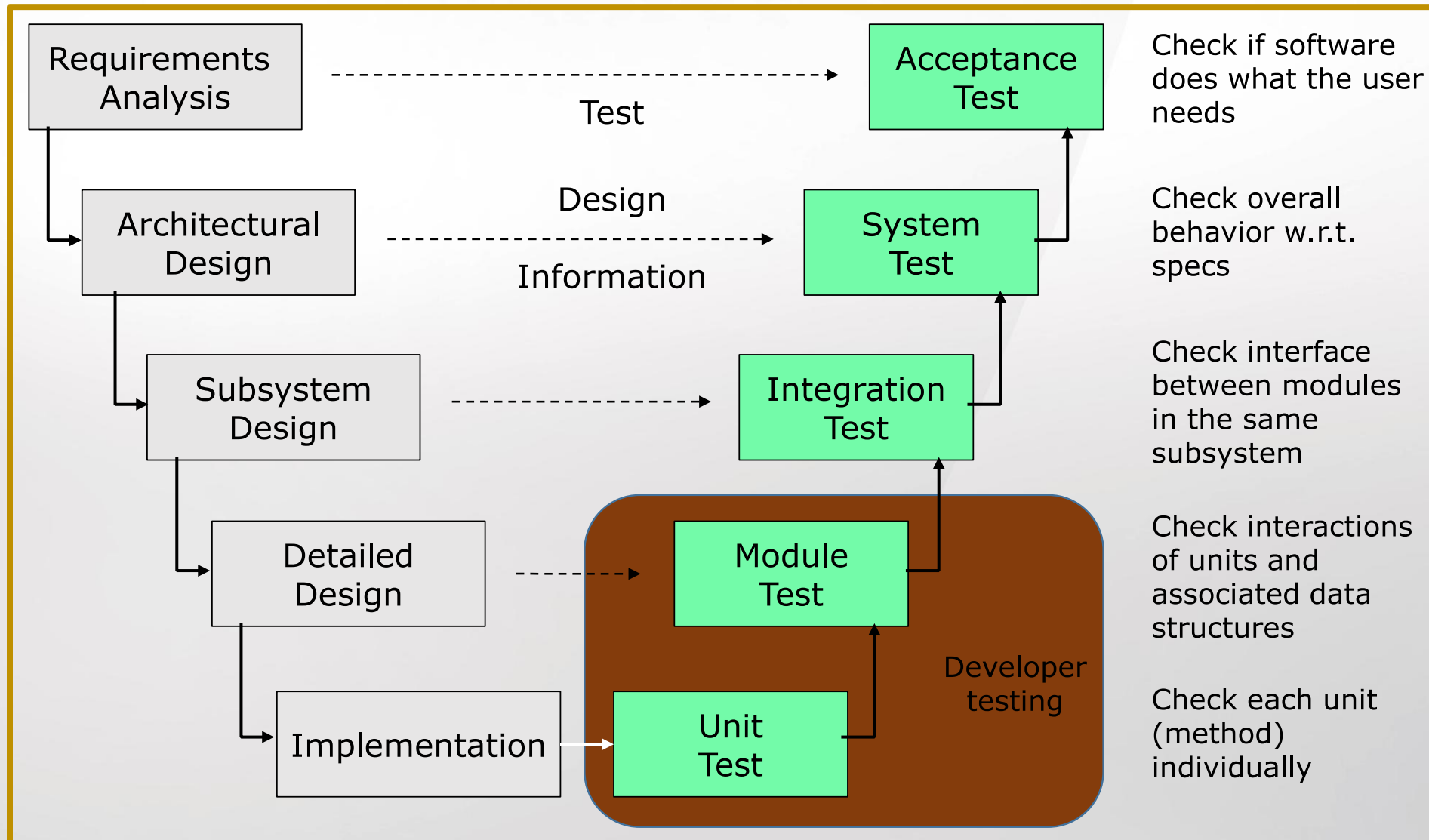
O objetivo é **eliminar os defeitos mais cedo possível**

- ✓ Melhorar a qualidade
- ✓ Reduzir custos
- ✓ Preservar satisfação do usuário

- ✓ **Teste Estático:** testa sem executar o software
 - Realiza inspeção de software e análise (formal)
 - Exemplos: verificação formal, lint checkers
 - Vamos ver algumas ferramentas, mas não é o foco da disciplina

- ✓ **Teste dinâmico:** teste executando o programa com entradas reais
 - É o **foco** da disciplina

- ✓ **Validação**: realizado no final do processo de desenvolvimento **por usuários** para verificar se o software atende as necessidades
- ✓ **Verificação**: realizado **por desenvolvedores** durante o desenvolvimento do software para verificar se atende os requisitos



Se considerarmos os últimos 30 anos da computação...

O teste de software se tornou mais **fácil** e **difícil** do que nunca!

Difícil pois...

- Vasta gama de linguagens de programação
- Muitas plataformas de hardware
- Diversos Sistemas Operacionais
- Onipresença do software (altamente difundido na sociedade)

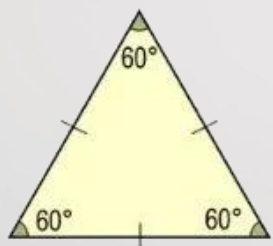
Fácil pois...

- Sistemas Operacionais modernos (serviços e rotinas bem testadas)
 - GUIs, por exemplo, facilmente desenvolvidas a partir de bibliotecas
- Necessidade reduzida de testar objetos pré-programados
- Inteligência Artificial para ajudar no desenvolvimento dos testes

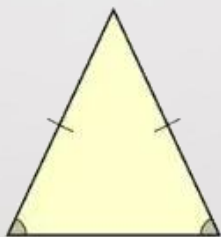
Agora, antes de começarmos... Vamos fazer um teste!

Escreva um conjunto de **casos de teste** – *conjuntos específicos de dados* – para testar adequadamente um programa relativamente simples:

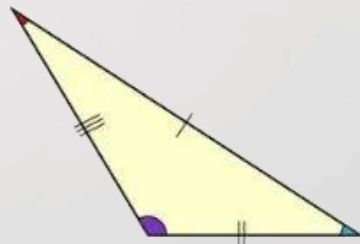
O programa recebe três valores inteiros. Os três valores representam os comprimentos dos lados de um triângulo. O programa exibe uma mensagem informando se o triângulo é escaleno, isósceles ou equilátero.



Equilátero



Isósceles



Escaleno

Lembre-se de que um triângulo **escaleno** é aquele em que não há dois lados iguais, enquanto um triângulo **isósceles** tem dois lados iguais e um triângulo **equilátero** tem três lados de comprimento igual. Além disso, os ângulos opostos aos lados iguais num triângulo isósceles também são iguais, e todos os ângulos num triângulo equilátero são iguais.

Vamos avaliar as respostas (1 ponto para cada “sim”):

1) Você tem um caso de teste que representa um triângulo escaleno válido?

(Observe que casos de teste como 1, 2, 3 e 2, 5, 10 não garantem uma resposta sim porque um triângulo com essas dimensões não é válido.)

2) Você tem um caso de teste que representa um triângulo equilátero válido?

3) Você tem um caso de teste que representa um triângulo isósceles válido?

(Observe que um caso de teste representando 2, 2, 4 não contaria porque não é um triângulo válido.)

4) Você tem pelo menos três casos de teste que representam triângulos isósceles válidos, de modo que você tentou todas as três permutações de dois lados iguais?

(Por exemplo: 3, 3, 4; 3, 4, 3; 4, 3, 3.)

5) Você tem um caso de teste em que um lado tem valor zero?

6) Você tem um caso de teste em que um lado tem um valor negativo?

7) Você tem um caso de teste com três inteiros maiores que zero, tal que a soma de dois números é igual ao terceiro?

(Ou seja, se o programa dissesse que 1, 2, 3 representa um triângulo escaleno, conteria um bug.)

Vamos avaliar as respostas (1 ponto para cada “sim”):

8) Você tem um caso de teste com três números inteiros maiores que zero, de modo que a soma de dois números seja menor que o terceiro?

(Como 1, 2, 4 ou 12, 15, 30)

9) Você tem pelo menos três casos de teste na categoria 9, de modo que tenha tentado todas as três permutações?

(Por exemplo, 1, 2, 4; 1, 4, 2; e 4, 1, 2)

10) Você tem um caso de teste em que todos os lados são zero (0, 0, 0)?

11) Você tem pelo menos um caso de teste especificando valores não inteiros?

12) Você tem pelo menos um caso de teste especificando o número errado de valores?

13) Para cada caso de teste você especificou a saída esperada do programa além dos valores de entrada?

É claro que um conjunto de casos de teste que satisfaça essas condições não garante que você encontrará todos os erros possíveis!

Entretanto, as questões de 1 a 13 representam erros que realmente ocorreram em diferentes versões deste programa, um teste adequado deste programa deve expor pelo menos esses erros.

E então, qual foi sua pontuação???

Programadores profissionais altamente qualificados pontuam, em média, apenas **7,8/14**.

[The Art of Software Testing]

- ✓ O objetivo do teste que fizemos é ilustrar que testar um programa trivial como este já não é uma tarefa fácil.
- ✓ Dado que isto é verdade, imaginemos a dificuldade de testar um sistema de controle de tráfego aéreo com 100.000 requisitos, um compilador ou mesmo um programa de folha de pagamento.
- ✓ Ao trabalhar com este exemplo, pode parecer assustador testar minuciosamente um programa complexo do mundo real.

Embora a tarefa possa ser assustadora, o teste adequado de programas é uma parte **muito necessária** — e **alcançável** — do desenvolvimento de software, como veremos nesta disciplina.

- ✓ Consiste em chamar uma função ou método de uma classe (*unit*) e executá-la com um conjunto de parâmetros conhecidos.
- ✓ O resultado é então comparado com o resultado esperado.

- ✓ Se essa expressão *assert* for avaliada como 0, isso causará uma falha (*assertion failure*) que encerrará o programa.

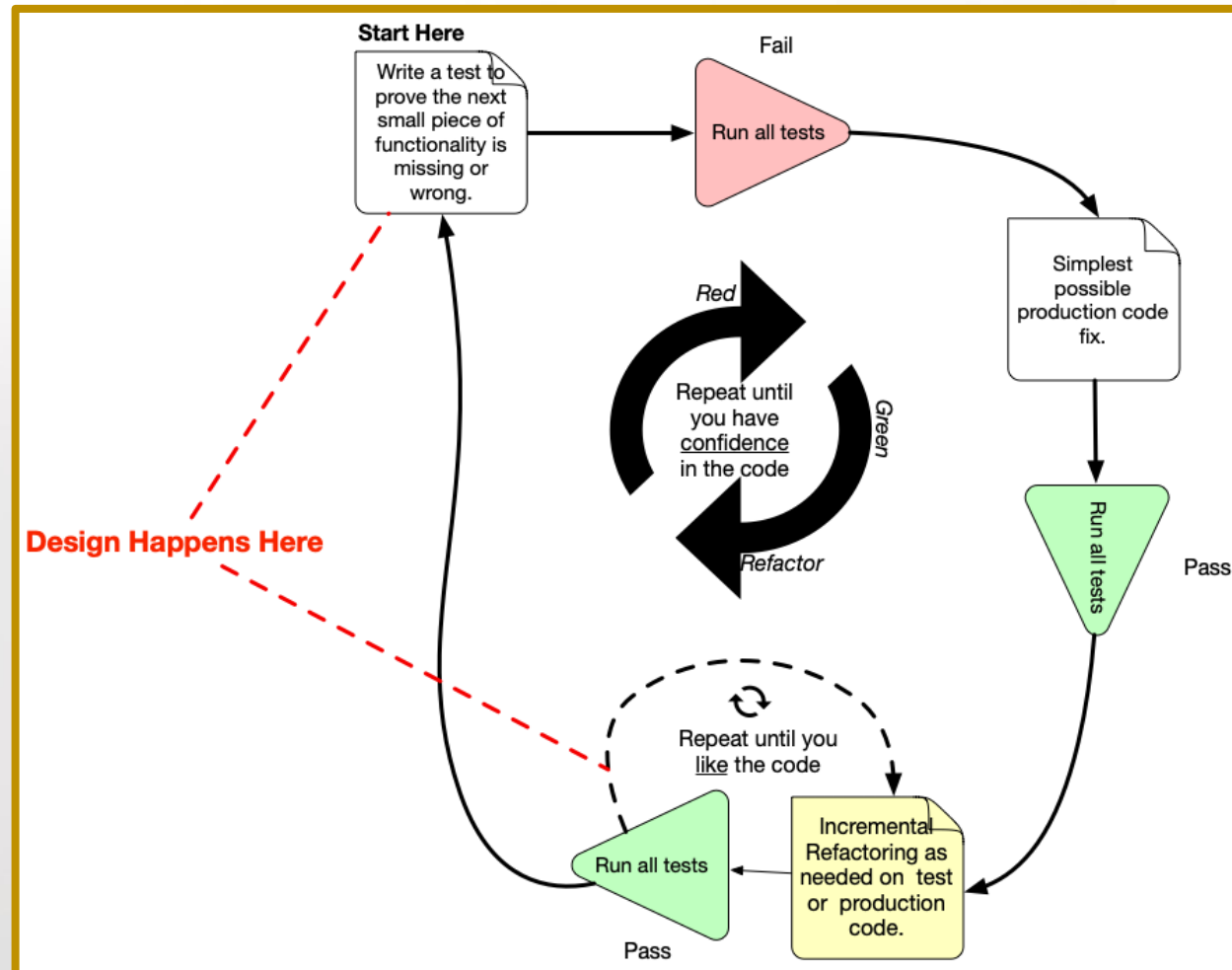
```
int foo(int a, int b) {  
    return a-b-1;  
}  
  
void test_foo() {  
    int ret = foo(1, 2);  
    assert(ret == -2, "wrong value returned")  
}
```

- ✓ Segundo o livro, os testes devem ser **FIRST**:
 - **Fast**: Os testes devem ser rápidos, rápidos o suficiente para que os desenvolvedores os executem em cada pequena alteração, sem esperar a fim de não interromperem o fluxo do projeto.
 - **Isolated**: Os testes devem ser isolados. Um teste não deve impactar o próximo teste. Os testes também isolam as falhas.
 - **Repeatable**: Os testes são repetíveis, repetível significa automatizado. Os testes são executados em um loop sempre dando o mesmo resultado.
 - **Self-verifying**: Os testes verificam seus resultados, relatando um simples "OK" quando são aprovados, enquanto fornecem detalhes concisos quando eles falham.
 - **Timely**: Os testes são feitos a tempo. Os programadores escrevem os testes em uma etapa diferente, isto é, normalmente ele é escrito um pouco antes do código de produção, evitando bugs.

- ✓ É uma técnica para desenvolvimento de software incremental
- ✓ Filosofia:
 1. Desenvolva o teste antes
 2. Execute o teste, gerando falhas
 3. Só depois desenvolva o código até que os testes passem
 4. Repita o processo para novas funcionalidades
- ✓ O teste representa o comportamento desejado do código
- ✓ A cada mudança no código, execute novamente os testes

- A chave é começar pequeno. Testes simples.
- Aumente seu código lentamente.
- Só torne seu código mais complexo se os testes exigirem isso.
- Pense em uma boa sequência de testes

✓ Máquina de Estados do TDD:



- ✓ É um software que permite o programador expressar o comportamento dos testes
- ✓ Deve possuir:
 - Uma linguagem para descrever os testes e valores esperados
 - Mecanismo para executar os testes
 - Relatórios dos teste

- ✓ Framework para Teste de Unidade em C
 - <https://github.com/ThrowTheSwitch/Unity>
- ✓ Motivação:
 - Pouca dependência de bibliotecas
 - Pode ser utilizado para testar Sistemas Embarcados - **Baremetal**, isto é (*sem OS*)
- ✓ Livro: James W. Grenning. ***Test-Driven Development for Embedded C***. 2011.
 - Capítulo 3
 - Fonte: https://pragprog.com/titles/jgade/source_code

- ✓ Abra o terminal e vá para o diretório onde você vai guardar o projeto
- ✓ Execute:
 - git clone <https://github.com/iacanaw/Unity.git>
 - cd Unity/examples/foo
 - make all

✓ **sprintf**

- Compõe uma string com o mesmo texto que seria impresso se fosse usado em printf, mas em vez de ser impresso, o conteúdo é armazenado como uma string C no buffer apontado pelo 1º argumento.
- Retorna o número de bytes que foi copiado para essa string!

- ✓ TEST (<group-of-test>, <test-name>)
- ✓ TEST_ASSERT_EQUAL – compara 2 inteiros
- ✓ TEST_ASSERT_EQUAL_STRING – compara string C

```
TEST(sprintf, NoFormatOperations)
{
    char output[5] = "";
    TEST_ASSERT_EQUAL(3, sprintf(output, "hey"));
    TEST_ASSERT_EQUAL_STRING("hey", output);
}
```

- ✓ E se sprintf usar mais de 12 bytes ?

```
TEST(sprintf, InsertString)
{
    char output[20] = "";

    TEST_ASSERT_EQUAL(12, sprintf(output, "Hello %s\n", "World"));
    TEST_ASSERT_EQUAL_STRING("Hello World\n", output);
}
```

- ✓ Esse teste verifica bytes após a string

```
TEST(sprintf, InsertString)
{
    char output[20];
    memset(output, 0xaa, sizeof output); ← Copio 0xaa em todas
                                           as posições do output

    TEST_ASSERT_EQUAL(12, sprintf(output, "Hello %s\n", "World"));
    TEST_ASSERT_EQUAL_STRING("Hello World\n", output);
    TEST_ASSERT_BYTES_EQUAL(0xaa, output[13]); ← Faço a verificação se as outras
                                                  posições não foram alteradas
}
```

- ✓ Evitando duplicação de código entre os testes

```
TEST_GROUP(sprintf);

static char output[100];
static const char * expected;

TEST_SETUP(sprintf)
{
    memset(output, 0xaa, sizeof output);
    expected = "";
}

TEST_TEAR_DOWN(sprintf)
{
}
```

- ✓ Evitando duplicação de código entre os testes

```
static void expect(const char * s)
{
    expected = s;
}

static void given(int charsWritten)
{
    TEST_ASSERT_EQUAL(strlen(expected), charsWritten);
    TEST_ASSERT_EQUAL_STRING(expected, output);
    TEST_ASSERT_BYTES_EQUAL(0xaa, output[strlen(expected) + 1]);
}
```

- ✓ Evitando duplicação de código entre os testes

```
TEST(sprintf, NoFormatOperations)
{
    expect("hey");
    given(sprintf(output, "hey"));
}

TEST(sprintf, InsertString)
{
    expect("Hello World\n");
    given(sprintf(output, "Hello %s\n", "World"));
}
```

✓ Agrupando testes

```
#include "unity_fixture.h"

TEST_GROUP_RUNNER(sprintf)
{
    RUN_TEST_CASE(sprintf, NoFormatOperations);
    RUN_TEST_CASE(sprintf, InsertString);
}
```

✓ Função *main* do teste

```
#include "unity_fixture.h"

static void RunAllTests(void)
{
    RUN_TEST_GROUP(sprintf);
}

int main(int argc, char * argv[])
{
    return UnityMain(argc, argv, RunAllTests);
}
```

✓ *Unity output sem erros*

```
make
compiling SprintfTest.c
Linking BookCode_Unity_tests
Running BookCode_Unity_tests
..
-----
2 Tests 0 Failures 0 Ignored
OK
```

✓ *Unity output com erros*

```
TEST(sprintf, NoFormatOperations)
{
    char output[5];

    TEST_ASSERT_EQUAL(4, sprintf(output, "hey"));
    TEST_ASSERT_EQUAL_STRING("hey", output);
}
```



```
make
compiling SprintfTest.c
Linking BookCode_Unity_tests
Running BookCode_Unity_tests
..
TEST(sprintf, NoFormatOperations)
    stdio/SprintfTest.c:75: FAIL
    Expected 4 Was 3
-----
2 Tests 1 Failures 0 Ignored
FAIL
```

ATIVIDADE

1. Fazer o clone do repositório **Unity**
 - Referência: <https://github.com/iacanaw/Unity.git>
2. Replicar a pasta foo e adaptar para a *função dos triângulos*.
 - **cd Unity/examples**
 - **cp -fr foo triangulos**
3. Implementar os **testes unitários** para a função *função dos triângulos*
 - O arquivo **UnityAssertionsReference.md** em **Unity/docs** apresenta referências à todas as *assertions* disponíveis.